BUSINESS APPLICATIONS OF AI

Module 6 — Vibe Coding

Palm Beach Atlantic University · Graduate Program

Vibe Coding

> *"The bottleneck in building software is no longer typing. It's knowing what to build."*

Learning Objectives

By the end of this chapter, you will be able to:

- Define vibe coding accurately — including what Karpathy meant and the common misreading — and explain the three core competencies it requires.
- Apply the five-step specification-driven development process to a real product build.
- Select the right AI coding tool for a given task using the tool selection matrix.
- Assemble the modern founder MVP stack (Next.js + Supabase + Vercel + Stripe + PostHog) and explain why AI agents know it best.
- Identify and avoid the five most common vibe coding failure modes before they cost you users, money, or both.
- Implement basic secrets management and code review discipline in an agentic development workflow.

6.1 What Vibe Coding Actually Is

In early 2025, Andrej Karpathy — one of the founding researchers behind modern large language models — posted a message that ricocheted through the developer internet. He described a new programming mode he'd been experimenting with: one where he no longer read the code at all. He wrote intentions in natural language, accepted what the model generated, and kept going. He called it "vibe coding" — programming by feel, by outcome-specification, by vibes.

The internet took this and ran in the wrong direction.

Within weeks, Twitter was full of founders claiming they'd "vibe coded" a startup. Influencers posted ten-second clips of prompting Claude and watching a full application materialize. Venture blogs ran think-pieces about the death of the software engineer. Most of this missed the point entirely.

Here is what Karpathy actually described: a personal workflow, for his own exploratory throwaway projects, in which he didn't care about the code quality because he didn't intend to ship the code to real users. He was explicit about this. He said so. The people who turned "vibe coding" into a founder gospel either didn't read the whole post or chose to ignore the disclaimer in the middle.

What vibe coding actually represents — properly understood — is a genuine paradigm shift in how software gets built. Natural language has become the primary programming interface. You describe what you want; the agent produces code that attempts to satisfy that description. The act of writing

individual lines of code has become optional, and for many tasks, counterproductive. But this creates new demands, not fewer.

The Three Competencies

Effective vibe coding demands exactly three things that it does not teach you automatically:

1. Specifying. Writing clear, unambiguous, testable descriptions of what the software should do. This is harder than writing code. When you write code, ambiguity crashes the program immediately. When you write a prompt, ambiguity produces plausible-looking code that fails in subtle ways three weeks after launch. The discipline of specification — product requirements documents, user stories, acceptance criteria, edge case enumeration — is now the highest-leverage skill a founder can develop.

2. Reviewing. Reading code you did not write, quickly and accurately, at the level of a diff. Not understanding every line — that standard is unnecessary and inefficient. But understanding what changed, why it changed, and whether it could fail. Founders who skip this step ship security vulnerabilities, data loss bugs, and broken payment flows. Founders who master it ship fast and safely.

3. Integrating. Knowing how pieces fit together. An agent can write a Stripe webhook handler. It can write a Supabase query. It cannot automatically know that the webhook handler needs to update the Supabase row atomically, or that your existing middleware expects a specific header format. Integration — the work of fitting generated pieces into a coherent system — still requires a human brain with end-to-end context.

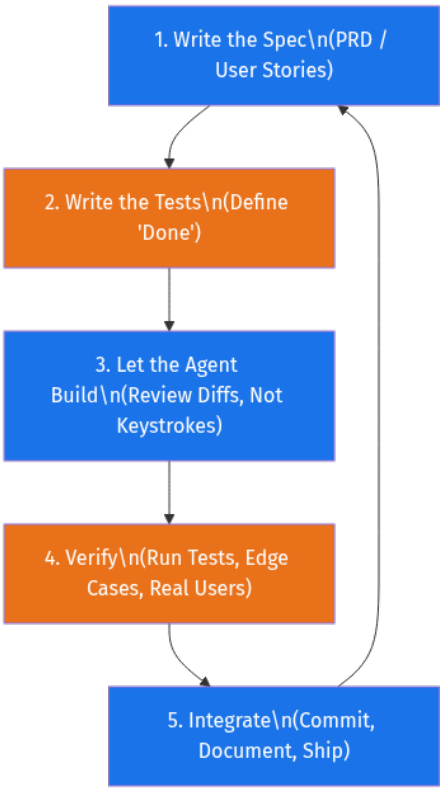
These three competencies are teachable. This chapter teaches them.

6.2 Specification-Driven Development

The single most common mistake in vibe coding is starting with the agent. The second most common is not having a definition of "done" before the build begins. Specification-driven development (SDD) is the antidote to both.

SDD is not a new concept. It borrows from test-driven development, behavior-driven development, and product management practice. What is new is how essential it has become in an agentic world. When the agent can produce hundreds of lines of code per minute, the constraint is no longer generation — it's direction. Without a spec, you are not coding with AI; you are wandering with AI.

The five steps of SDD:



Step 1: Write the Spec

The product requirements document (PRD) comes first. Always. Before you open any AI coding tool, you should have a written document that answers:

- What problem does this solve, for whom, and why now?
- What are the user stories? ("As a [user], I want to [action] so that [outcome].")
- What are the acceptance criteria for each story? These are the conditions that must be true for the feature to be "done."
- What are the out-of-scope items? Explicit non-features prevent scope creep.
- What are the edge cases worth naming? Not every edge case — the ones that could cause data loss, security failures, or broken payments.

A PRD does not need to be a 30-page document. For a startup MVP, two to four pages is standard. The discipline is in writing it before building, not in hitting a page count.

Step 2: Write the Tests

Before any agent writes a line of implementation code, you should have at least a skeleton of test cases. These can be unit tests, integration tests, or simple checklists — the format matters less than the act of defining "done" in testable terms.

Why tests before build? Because agents optimize for passing tests. When you give an agent a failing test suite alongside a spec, it has a concrete, machine-checkable definition of success. The output

quality improves substantially. This is the vibe coding equivalent of giving a contractor architectural drawings plus a building inspection checklist: they build to specification because the specification is measurable.

Step 3: Let the Agent Build

With a spec and tests in hand, you can now open your AI coding tool and begin building. The key mindset shift: you are reviewing diffs, not keystrokes. You are not watching the agent type and approving line by line. You are reading the output of each agent action — what changed, what was created, what was deleted — and asking: does this match the spec? Does it pass the tests? Could it fail in an unexpected way?

Resist the urge to take over and start writing code manually when the agent produces something imperfect. Instead, refine the prompt. The agent's output is a draft; your job is to be the editor, not the rewriter.

Step 4: Verify

Run the tests. Run the app. Log in as a test user and attempt to break it. Submit forms with invalid data. Try to access routes you shouldn't be able to access. Hit the API endpoints without authentication. This is the step most founders skip in the excitement of seeing something working, and it is the step that determines whether "working demo" becomes "working product."

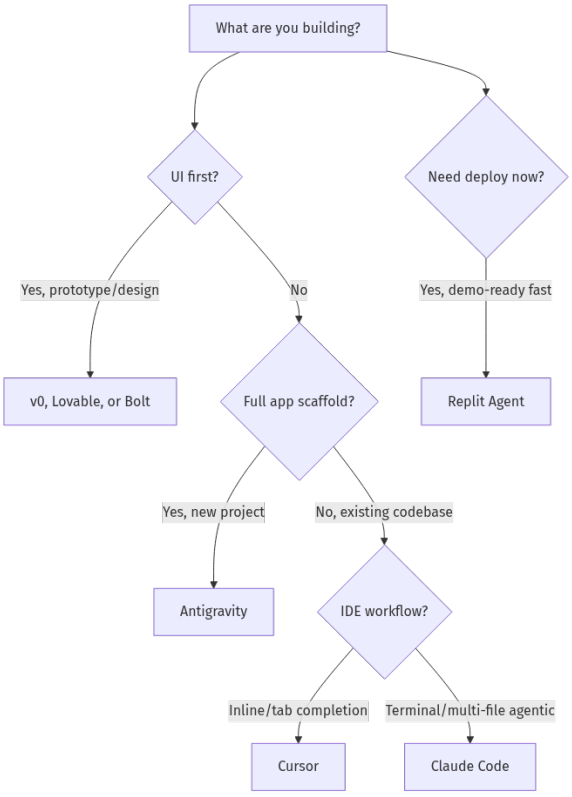
Edge case testing is not optional. The canonical founder failure — shipping a demo-only MVP that collapses under the second user — almost always traces to this step being skipped.

Step 5: Integrate, Commit, Document, Ship

Once verified, integrate the change into your main codebase, commit with a clear message, update the README if anything changed about how to run or deploy the project, and ship. Documentation is not bureaucracy — it is the artifact that allows you, your co-founder, or an agent running in a future session to understand what exists and why. A codebase without documentation is a codebase that only one person can maintain.

6.3 The Tool Selection Matrix

Not all AI coding tools are the same. They differ substantially in their interfaces, their strengths, and the contexts where they produce the best results. Choosing the wrong tool for a task doesn't just slow you down — it can produce structurally flawed output that requires a full rewrite.



Claude Code is a terminal-based agentic coding environment. You run it from the command line inside your existing project directory. It can read, write, and edit multiple files, run shell commands, run tests, and iterate on its own output. Claude Code excels at:

- Refactoring existing codebases — it can read the full context of a large project and make coordinated changes across many files.
- Iteration on complex logic — when the task requires multiple passes, reading error output, and self-correcting.
- Agentic multi-file edits — features that span components, routes, database schemas, and API handlers simultaneously.

Claude Code is the choice when you have an existing codebase and a well-specified task. It is not the fastest path to a first scaffold from nothing.

::

::

::

::

6.4 The MVP Stack for Founders

One of the underappreciated advantages of the modern founder stack is that AI agents know it extraordinarily well. The combination of Next.js, Tailwind CSS, shadcn/ui, Supabase, Vercel, Stripe, and PostHog has been written about, documented, and trained on more than almost any other stack in existence. When you ask Claude Code or Antigravity to build something using this stack, you get better output, faster, with fewer hallucinations than if you specified a more exotic combination of tools.

This is a real selection criterion. The best stack for a solo founder or small team is not necessarily the most technically optimal stack — it is the stack that the agents building it know best.

The Stack, Layer by Layer

Frontend: Next.js + Tailwind CSS + shadcn/ui

Next.js is the React framework that handles routing, server-side rendering, API routes, and deployment configuration. Tailwind provides utility-first CSS that produces consistent, responsive layouts without writing custom stylesheets. shadcn/ui provides a library of accessible, composable UI components that look professional without requiring a designer.

Why this combination? Because virtually every AI coding agent produces excellent Next.js + Tailwind + shadcn code. The documentation is extensive, the patterns are well-established, and the community output that agents were trained on is large and high-quality.

Backend and Database: Supabase

Supabase is an open-source Firebase alternative that provides a PostgreSQL database, authentication, file storage, and real-time subscriptions, all behind a clean JavaScript SDK. For a solo founder, it eliminates the need to set up a separate auth service, a separate database, and a separate storage solution. One Supabase project handles all three.

Supabase's Row Level Security (RLS) is worth understanding early: it allows you to define data access policies at the database level, so that users can only read and write their own data, without writing complex application-layer authorization logic.

Deployment: Vercel

Vercel is the deployment platform built by the creators of Next.js. It provides zero-configuration deployment (push to GitHub, live in seconds), automatic preview deployments for every pull request, and a generous free tier for early-stage applications. For Next.js applications, Vercel is the default choice.

Payments: Stripe

Stripe handles payment processing, subscription billing, invoicing, and tax compliance. For SaaS applications, Stripe's subscription API and webhook system are the standard. Agents know Stripe's API well; the patterns for creating checkout sessions, handling webhook events, and managing subscription states are well-documented and frequently generated correctly on the first attempt.

Analytics: PostHog

PostHog is an open-source product analytics platform that tracks user behavior, runs A/B tests, manages feature flags, and records session replays. It is self-hostable but also available as a cloud service with a generous free tier. For early-stage products, PostHog answers the question every founder needs to answer: are users actually using what I built, and where do they stop?

6.5 Common Failure Modes

Vibe coding failures are predictable. They fall into five patterns that appear in almost every failed AI-assisted build. Name them before they name you.

Failure Mode 1: The Demo-Only MVP

The demo-only MVP looks like a real product and works for exactly one user under exactly the conditions the demo was prepared for. Add a second user and the whole thing falls apart: shared state, hardcoded IDs, no authentication, database tables that assume a single record per table. This failure mode is almost universal when founders skip specification-driven development and go straight to "build me an app."

The fix: write user stories that explicitly include multi-user scenarios. User A and User B should not be able to see each other's data. Test by creating two accounts.

Failure Mode 2: Over-Engineered First Versions

Agents, when prompted vaguely, default to enterprise patterns. Ask for "a way to manage users" and you might get a full role-based access control system with permission matrices, audit logs, and a separate admin dashboard — when what you needed was a simple boolean is_admin column. Over-engineering a first version creates complexity that slows every subsequent iteration, confuses future agents (and future you), and delays the shipping of features users actually need.

The fix: explicitly constrain scope in every prompt. "Build the simplest possible version of X that satisfies these specific requirements."

Failure Mode 3: Under-Tested Agent Output in Production

Agent output is optimistic. It compiles. It runs the happy path. It does not reliably handle network failures, database constraint violations, invalid input from real users, or race conditions. Shipping unreviewed agent output directly to production — especially for payment flows, authentication, or data mutation — is a recipe for data loss or security failure.

The fix: step four of SDD, every time. Run tests. Run edge cases. Never skip verification.

Failure Mode 4: Scope Creep Spiral

The scope creep spiral begins the moment you add a feature to the prompt without first checking it against the PRD. "While you're in there, can you also add..." is how MVPs become six-month projects. Each new feature is faster to add than the last (because the foundation exists), which creates the illusion that adding features is free. It is not. Complexity compounds. Agents lose context. Bugs multiply.

The fix: maintain a ruthless backlog. Features that are not in the current sprint's spec go in the backlog. They do not go in the prompt.

Failure Mode 5: Trusting the Vibe Without Reading the Diff

This is Karpathy's failure mode — the one he acknowledged was only acceptable for throwaway personal projects. Accepting agent output without reading the diff means accepting whatever the agent decided to do, including deleting files it deemed redundant, changing database schemas without migrations, and introducing hardcoded values that will fail in production.

The fix: always read the diff. Always. Every single commit. This is non-negotiable.

6.6 Security and Responsibility

AI-generated code introduces security risks that are qualitatively different from handwritten code risks. When a developer makes a security mistake, it is usually because they didn't know the secure pattern. When an agent makes a security mistake, it is often because the prompt didn't specify the security requirement — and the agent produced functional code that achieved the stated goal while ignoring the unstated one.

Secrets Management

Never put secrets in prompts. Never put secrets in code files that you commit to version control. Never put secrets in comments. An API key in a prompt is an API key that has passed through a third-party model provider's API. An API key in a committed file is an API key that is now in GitHub's search index.

The correct pattern:

- Store secrets in environment variables on your deployment platform (Vercel's environment variable settings, Supabase's vault).
- Access them in code via process.env.VARIABLE_NAME.
- Use a .env.local file locally, and ensure .env.local is in .gitignore — it should be there by default in any Next.js project, but verify.
- When asking an agent to work with integrations that require API keys, reference them by variable name (process.env.STRIPESECRETKEY) in the prompt, not by value.

The Agent with Production Access Problem

Some AI coding tools can be given direct access to production systems — production databases, live API keys, deployment pipelines. This is convenient and dangerous in equal measure. An agent with production access can drop a database table. It can deplete API credits. It can send emails to real users.

Best practice: agents work against development and staging environments. Production access requires explicit human action. This is not a limitation of current AI capability — it is a permanent professional practice.

Code Review Discipline

Code review discipline, when you didn't write the code, means answering four questions before accepting any agent-generated change:

- Does this do what the spec says it should do? Read the acceptance criteria from your PRD and verify the diff actually implements them. Agents sometimes produce code that handles the happy path described in the prompt while silently ignoring edge cases that were in the spec.
- Does this introduce any credentials, keys, or sensitive data in plaintext? Search the diff for strings that look like API keys, passwords, database connection strings, or tokens. Agents occasionally hardcode values from the conversation context — if your prompt mentioned an API key by value, the agent may have used it literally in the code.
- Does this modify any database schema without a reversible migration? Schema changes without migrations mean you cannot roll back without data loss. Every ALTER TABLE, DROP COLUMN, or CREATE TABLE statement should be accompanied by a reversible migration file.
- Does this add a dependency I haven't evaluated? (New npm install calls can introduce supply chain vulnerabilities.) Check package.json changes in the diff. Each new dependency is a decision — it has a license, a maintenance status, and a potential security surface area. A two-minute check on the package's npm page and recent GitHub commits is not excessive due diligence.

These four questions take two to five minutes per diff. They prevent the majority of agentic security incidents and the majority of "it worked in staging" deployment failures. Build the habit before the stakes are real.

Licensing and Attribution

AI-generated code has unresolved licensing questions. The legal landscape is evolving rapidly, and any specific ruling cited here may be superseded within the publication window of this textbook. The practical guidance:

- Review your AI coding tool's terms of service with respect to IP ownership. Most major tools (Claude Code, Cursor, Copilot) assert that you own the output, but verify for your jurisdiction and use case.
- For enterprise or regulated applications, work with counsel to understand exposure before shipping AI-generated code at scale.
- Keep a record of significant architectural decisions and which tools contributed to them — not for legal attribution, but for your own understanding of what you'd need to maintain or rewrite if a tool changed its terms.

Case Study: Zero to SaaS in One Lecture

The following is a condensed transcript of a live build conducted by the instructor during a graduate class. The application built was a simple waitlist manager: businesses sign up, collect emails from potential customers, and get analytics on waitlist growth. The entire build — from blank repository to deployed URL with authentication, database, and a functional waitlist form — took 47 minutes.

Minutes 0–12: The PRD

Before opening any tool, I wrote the product requirements document. I shared my screen and wrote it live in a plain text file. The audience could see me make decisions and cross things out. The PRD covered:

- Two user types: business owners and waitlist subscribers
- Business owners could create a waitlist page with a custom slug (e.g., waitlist.app/my-startup)
- Subscribers could join the waitlist with email and name
- Business owners could see a count and list of subscribers
- Out of scope: email verification, custom branding, multiple waitlists per account

Minutes 12–18: The Test Skeleton

I wrote six acceptance criteria in plain English, then asked Claude Code to turn them into Jest test skeletons with expect calls. I did not implement the tests — I just established the structure. This took six minutes.

Minutes 18–31: The Scaffold with Antigravity

I opened Antigravity and pasted the PRD. I specified: Next.js, Tailwind, shadcn, Supabase, Vercel. I asked for the full application scaffold with auth wired up and the database schema created. Antigravity produced a complete project in four minutes. I reviewed the diff. The schema was almost right — it had the right tables but was missing a unique constraint on the slug column. I flagged this in a follow-up prompt: "Add a unique constraint on the slug column of the waitlist_pages table." Corrected in 30 seconds.

Minutes 31–40: The Waitlist Form with Claude Code

I opened Claude Code in the project directory and asked it to build the public-facing waitlist form component and the API route that handles form submission. I referenced the PRD explicitly in the prompt. Claude Code produced both in one pass. I read the diff. The API route was not validating email format before inserting — I asked for email validation. Fixed in one pass.

Minutes 40–44: Deployment to Vercel

git push to a GitHub repository I had created before class. Connected the repository to a Vercel project. Set the three required environment variables (Supabase URL, Supabase anon key, Supabase service role key). Deployed. Live URL within 90 seconds of the first git push.

Minutes 44–47: The Wrong Turns

Here is what I did not show in real time but will now: I had three failed prompts during the Claude Code phase. The first attempt at the waitlist form produced a client component that tried to call Supabase directly from the browser using the service role key — a critical security error that would have exposed full database access to any user who opened the network inspector. I caught it in the diff review. The second attempt fixed the key issue but broke the form validation. The third attempt was correct. The agent's first output was not the output I shipped. This is normal. This is why you read the diff.

The wrong turns are the lesson. A live build without wrong turns is a rehearsed demo, not a real build. The honest count: three failed prompts, one caught security error, one schema fix, and one moment where I almost accepted an over-engineered solution (the agent produced a full email verification flow when the spec explicitly said email verification was out of scope). I caught it

because I had the PRD open in a second window. The spec is not just a planning document — it is an active defense against scope creep and agent drift throughout the entire build.

If you take one thing from this case study: write the spec first, keep it visible, and read every diff. The agent is a fast and capable collaborator. It is not a careful one.

Faith Integration — Module 6

> *"In the beginning God created the heavens and the earth." — Genesis 1:1 (NIV)*

> *"So God created mankind in his own image, in the image of God he created them." — Genesis 1:27 (NIV)*

Imago Dei: Humans as Sub-Creators

The very first thing Scripture tells us about God is that he creates. And the very first thing it tells us about humanity is that we are made in his image — the image of a Creator. This is the doctrine of the Imago Dei, and it has radical implications for what it means to build software.

When you use Claude Code to translate a business idea into a functioning application, you are doing something uniquely human: taking what exists only in the mind and making it real in the world. Every other creature in the Genesis narrative is created; only humans are commissioned to be co-creators — to continue the work of bringing order, beauty, and function into the world through their own acts of making.

Vibe coding does not diminish this calling. It amplifies it. For the first time in history, the barrier between "I can imagine this" and "I can build this" has been reduced to the quality of your thinking and the clarity of your communication. The non-developer can now build. The solo founder can now ship what previously required a team. This is an extraordinary gift — and like all gifts, it carries responsibility.

What will you build? For whom? Toward what end? The Imago Dei does not just celebrate the act of creation; it anchors it in the character of the Creator — a God who builds things that are good, that serve, that flourish. Your work with code is not exempt from that standard.

' þ Faith Integration Paper — Module 6

Assignment: Using Claude or Gemini as a co-writer, compose a 1-page reflection paper (approximately 300–400 words) responding to the following prompt:

> *Genesis 1 teaches that you are made in the image of a Creator God — you are, by design, a maker. Reflect on the experience of building your MVP in this module. What does it feel like to create something from nothing? How does the concept of Imago Dei change the way you think about the responsibility you carry as a builder? What does it mean to build software that is "good" in the biblical sense — not just technically functional, but truly beneficial for the people who will use it?*

Process:

- Co-draft with Claude or Gemini, sharing the prompt and your build experience.
- Revise with specific details from your MVP work — what did you actually make?
- Submit the human-owned version.

Format: 1 page, double-spaced, 12pt font, Times New Roman. Include Genesis 1:1 and 1:27 at the top.

Grading: Theological depth, personal reflection on the building experience, specificity about your actual MVP.

Lab 6: Ship Your MVP

Deliverable: A live, deployed URL for the product you pitched in Chapter 5.

Requirements

Your deployed MVP must include all of the following:

- Authentication — users can sign up, log in, and log out. User data is isolated per account.
- Database — at least one data entity that belongs to a user and persists across sessions.
- Transaction flow — at least one action that creates, modifies, or deletes a database record through the UI (not just a read).
- Basic analytics — PostHog (or equivalent) installed and capturing at least one custom event.
- Reproducible from README — a collaborator should be able to clone the repository, follow the README, and run the application locally without asking you for help.

Submission Checklist

- Live URL (must be accessible without any credentials from grader)
- GitHub repository URL (public or with grader access granted)
- One-paragraph explanation of the one feature you cut from scope to ship on time, and why you cut it
- Screenshot of PostHog dashboard showing at least one recorded event

Grading Rubric

Criterion	Points
Live URL accessible and functional	20
Auth working (sign up, login, logout)	20

Database persistence confirmed	20
Transaction flow functional	20
Analytics capturing data	10
README enables local reproduction	10
Total	100

AI Studio Build (Weekly): The Parallel Prototype

Using Google AI Studio's Build feature, rebuild one key screen or feature of your Chapter 6 MVP as a Gemini-powered shareable application.

The goal is not to replace your MVP. The goal is to produce a side-by-side comparison that answers a real product question: which tool was faster for which task?

Deliverables

- AI Studio Share Link — your rebuilt screen/feature as a shareable AI Studio app
- One-page tool-choice analysis — structured as follows: - What you built in each tool (one sentence per tool) - Time to first working output in each tool - Quality of first output (1–5 scale, with justification) - Which tool you would use for this task in a production context, and why - One thing each tool does that the others cannot

Capability introduced: comparative prototyping and deliberate tool selection. The professional vibe coder does not have a single favorite tool — they have a calibrated matrix of tool-to-task fit that they refine with every build.

Discussion Prompts

Prompt 1: The Skill Displacement Question

Vibe coding reduces the barrier to building software. A non-programmer can now ship a functioning web application. Traditional software engineering programs emphasize algorithmic thinking, data structures, system design, and low-level implementation skill.

Your post (400–500 words): Drawing on at least two scholarly sources published no more than two years ago, argue a position on the following question: Does vibe coding make traditional software engineering skills more or less valuable for founders and product leaders? Your argument must engage with evidence — not just opinion. Consider the role of specification quality, security review,

integration complexity, and the difference between building a demo and maintaining a production system.

Peer responses: Respond to two peers whose position differs from yours. Responses must engage with the specific evidence they cited, not just restate your own position.

Prompt 2: The Responsibility Allocation Problem

An AI coding agent produces a payment processing function with a subtle bug. The bug causes some customers to be charged twice. The bug was not in the specification; the agent introduced it. The founder shipped the output without reading the diff.

Your post (400–500 words): Using at least two scholarly sources published no more than two years ago, analyze where responsibility lies in this scenario. Is the founder wholly responsible? Is the tool developer partially responsible? Does the answer change if the founder read the diff and missed the bug vs. didn't read it at all? What institutional or regulatory frameworks, if any, should govern AI-generated code in production applications?

Peer responses: Respond to two peers. At least one response must challenge an assumption in their argument — identify a condition under which their conclusion would not hold.

Readings

Required

- Karpathy, A. (2025, February). Vibe coding [Blog post / social post]. Read the original source, not the summaries. The summaries are wrong.
- Anthropic. (n.d.). Claude Code documentation. https://docs.anthropic.com/en/docs/claude-code
- Supabase. (n.d.). Row Level Security. https://supabase.com/docs/guides/auth/row-level-security
- Vercel. (n.d.). Next.js deployment documentation. https://vercel.com/docs/frameworks/nextjs

Recommended

- PostHog. (n.d.). Product analytics for startups. https://posthog.com/docs
- Stripe. (n.d.). Webhooks best practices. https://stripe.com/docs/webhooks/best-practices
- shadcn. (n.d.). Component library documentation. https://ui.shadcn.com/docs
- GitHub. (n.d.). Secret scanning. https://docs.github.com/en/code-security/secret-scanning

---

Glossary

Vibe Coding A natural-language-first approach to software development where the programmer specifies outcomes in prose rather than writing implementation code directly. The term was coined by Andrej Karpathy to describe exploratory personal projects; it has since been adapted as a professional methodology when combined with specification-driven development discipline.

Specification-Driven Development (SDD) A software development methodology that requires a written product requirements document and testable acceptance criteria before any implementation work begins. Particularly valuable in AI-assisted development, where agents optimize for passing stated requirements.

Product Requirements Document (PRD) A written document that defines what a software product should do, for whom, and under what conditions. The PRD precedes all implementation work in specification-driven development. It includes user stories, acceptance criteria, scope boundaries, and known edge cases.

Diff In version control, a diff is a display of the changes between one version of a file and another — lines added, removed, or modified. Reviewing the diff is the primary code review action in a vibe coding workflow.

Row Level Security (RLS) A PostgreSQL feature, prominently used in Supabase, that allows data access policies to be defined at the database level. With RLS enabled, a query automatically filters results to only rows the current user is permitted to see, preventing cross-user data leakage without application-layer logic.

Agent In the context of AI coding tools, an agent is an AI system that can take multi-step actions — reading files, writing files, running commands, observing output, and iterating — without requiring a human prompt for each individual action.

Scaffold A generated application skeleton that includes the project structure, configuration files, and boilerplate code needed to start development, without containing the application's actual business logic.

Agentic Build A software build process in which an AI agent autonomously performs a series of development actions — writing files, running tests, debugging failures — to produce a functional feature or application with minimal human intervention between steps.

Claude Code Anthropic's terminal-based agentic coding environment. Runs in the developer's existing project directory, can read and write files, execute shell commands, and iterate on its own output. Optimized for complex refactoring and multi-file agentic edits.

Antigravity An AI-native application scaffolding and build platform. Accepts natural language product descriptions and produces complete, deployable application scaffolds with integrations pre-wired.

Cursor An AI-native IDE built on VS Code. Provides inline AI assistance, tab-completion for code blocks, and a context-aware chat interface alongside traditional code editing.

shadcn/ui A collection of accessible, composable React UI components that can be copied into a project and customized. Designed to work natively with Tailwind CSS. Part of the standard modern founder MVP stack.

Supabase An open-source backend-as-a-service platform providing PostgreSQL database, authentication, file storage, and real-time capabilities behind a JavaScript SDK. The standard database and auth layer in the modern founder MVP stack.

PostHog An open-source product analytics platform providing event tracking, session recording, A/B testing, and feature flags. Used to understand how users actually interact with a deployed product.

Acceptance Criteria Specific, testable conditions that must be true for a feature to be considered complete. Written during Step 2 of specification-driven development. Acceptance criteria define

"done" in terms an automated test or a human reviewer can verify.

Supply Chain Attack A security attack that targets software dependencies rather than the application itself — for example, a malicious actor publishing a compromised version of a popular npm package that an application's build system will automatically install. A risk of adding new dependencies without evaluation.

Environment Variable A key-value pair stored outside the application code that provides configuration and secrets to the running application. The standard pattern for keeping API keys and database credentials out of version control.